

FreeRTOS基础教程第一章创建任务

1.硬件初始化

本章创建的任务需要用到[开发板](#)上的LED，所以先要将LED相关的函数初始化好，为了方便以后统一管理板级外设的初始化，我们在main.c文件中创建一个BSP_Init()函数，专门用于存放板级外设初始化函数，

```
1.
2. /*****
3.  * @ 函数名   :   BSP_Init
4.  * @ 功能说明:   板级外设初始化，所有板子上的初始化均可放在这个函数里面
5.  * @ 参数     :
6.  * @ 返回值   :   无
7.  *****/
8. static void BSP_Init(void)
9. {
10.     /*
11.      * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
12.      * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
13.      * 都统一用这个优先级分组，千万不要再分组，切忌。
14.      */
15.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
16.
17.     /* LED 初始化 */
18.     LED_GPIO_Config();
19.
20.     /* 串口初始化 */
21.     USART_Config();
22.
23. }
```



执行到BSP_Init()函数的时候，操作系统完全都还没有涉及到，即BSP_Init()函数所做的工作跟我们以前编写的裸机工程里面的硬件初始化工作是一模一样的。运行完BSP_Init()函数，接下来才慢慢启动操作系统，最后运行创建好的任务。有时候任务创建好，整个系统跑起来了，可想要的实验现象就是不出来，比如LED不会亮，串口没有输出，[LCD](#)没有显示等等。如果是初学者，这个时候就会心急如焚，四处求救，那怎么办？这个时候如何排除是硬件的问题还是系统的问题，这里面有个小小的技巧，即在硬件初始化好之后，顺便测试下硬件，测试方法跟裸机编程一样，

```
1.
2. /*****
3.  * @ 函数名   :   BSP_Init
4.  * @ 功能说明:   板级外设初始化，所有板子上的初始化均可放在这个函数里面
5.  * @ 参数     :
6.  * @ 返回值   :   无
7.  *****/
8. static void BSP_Init(void)
9. {
10.     /*
11.      * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
12.      * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
13.      * 都统一用这个优先级分组，千万不要再分组，切忌。
14.      */
15.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
16.
17.     /* LED 初始化 */
18.     LED_GPIO_Config();
19.
20.     /* 测试硬件是否正常工作 */
21.     LED1_ON;
22.
23.     /* 让程序停在这里，不再继续往下执行 */
24.     while(1);
25.
26.     /* 串口初始化 */
27.     USART_Config();
28.
29. }
```



2.创建单任务—SRAM静态内存

这里，我们创建一个单任务，任务使用的栈和任务控制块都使用静态内存，即预先定义好的全局变量，这些预先定义好的全局变量都存在内部的SRAM中。

2.1定义任务函数

任务实际上就是一个无限循环且不带返回值的C函数。目前，我们创建一个这样的任务，让开发板上面的LED灯以500ms的频率闪烁，

```
1.
2. /*****
3.  * @ 函数名   :   LED_Task
4.  * @ 功能说明:   LED_Task任务主体
5.  * @ 参数     :
6.  * @ 返回值   :   无
7.  *****/
8. static void LED_Task(void* parameter)
9. {
10.     while (1)
11.     {
```

```

12.     LED1_ON;
13.     vTaskDelay(500); /* 延时500个tick */
14.     printf("LED_Task Running, LED1_ON\r\n");
15.
16.     LED1_OFF;
17.     vTaskDelay(500); /* 延时500个tick */
18.     printf("LED_Task Running, LED1_OFF\r\n");
19. }
20. }

```



2.2 空闲任务与定时器任务堆栈函数实现

当我们使用了静态创建任务的时候，configSUPPORT_STATIC_ALLOCATION 这个宏定义必须为 1（在 FreeRTOSConfig.h 文件中），并且我们需要实现两个函数：vApplicationGetIdleTaskMemory() 与 vApplicationGetTimerTaskMemory()，这两个函数是用户设定的空闲（Idle）任务与定时器（Timer）任务的堆栈大小，必须由用户自己分配，而不能是动态分配，

```

1. /* 空闲任务任务堆栈 */
2. static StackType_t Idle_Task_Stack[configMINIMAL_STACK_SIZE];
3. /* 定时器任务堆栈 */
4. static StackType_t Timer_Task_Stack[configTIMER_TASK_STACK_DEPTH];
5.
6. /* 空闲任务控制块 */
7. static StaticTask_t Idle_Task_TCB;
8. /* 定时器任务控制块 */
9. static StaticTask_t Timer_Task_TCB;
10.
11. /**
12.  *****
13.  * @brief 获取空闲任务的任务堆栈和任务控制块内存
14.  *          ppxTimerTaskTCBBuffer : 任务控制块内存
15.  *          ppxTimerTaskStackBuffer : 任务堆栈内存
16.  *          pulTimerTaskStackSize : 任务堆栈大小
17.  * @date 2018-xx-xx
18.  *****
19.  */
20. void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer,
21.                                     StackType_t **ppxIdleTaskStackBuffer,
22.                                     uint32_t *pulIdleTaskStackSize)
23. {
24.     *ppxIdleTaskTCBBuffer=&Idle_Task_TCB; /* 任务控制块内存 */
25.     *ppxIdleTaskStackBuffer=Idle_Task_Stack; /* 任务堆栈内存 */
26.     *pulIdleTaskStackSize=configMINIMAL_STACK_SIZE; /* 任务堆栈大小 */
27. }
28.
29. /**
30.  *****
31.  * @brief 获取定时器任务的任务堆栈和任务控制块内存
32.  *          ppxTimerTaskTCBBuffer : 任务控制块内存
33.  *          ppxTimerTaskStackBuffer : 任务堆栈内存
34.  *          pulTimerTaskStackSize : 任务堆栈大小
35.  *****
36.  */
37. void vApplicationGetTimerTaskMemory(StaticTask_t **ppxTimerTaskTCBBuffer,
38.                                     StackType_t **ppxTimerTaskStackBuffer,
39.                                     uint32_t *pulTimerTaskStackSize)
40. {
41.     *ppxTimerTaskTCBBuffer=&Timer_Task_TCB; /* 任务控制块内存 */
42.     *ppxTimerTaskStackBuffer=Timer_Task_Stack; /* 任务堆栈内存 */
43.     *pulTimerTaskStackSize=configTIMER_TASK_STACK_DEPTH; /* 任务堆栈大小 */
44. }

```



2.3 定义任务栈

目前我们只创建了一个任务，当任务进入延时的时候，因为没有另外就绪的用户任务，那么系统就会进入空闲任务，空闲任务是 FreeRTOS 系统自己启动的一个任务，优先级最低。当整个系统都没有就绪任务的时候，系统必须保证有一个任务在运行，空闲任务就是为这个设计的。当用户任务延时到期，又会从空闲任务切换回用户任务。在 FreeRTOS 系统中，每一个任务都是独立的，他们的运行环境都单独的保存在他们的栈空间当中。那么在定义好任务函数之后，我们还要为任务定义一个栈，目前我们使用的是静态内存，所以任务栈是一个独立的全局变量，具体见代码清单 14-5。任务的栈占用的是 MCU 内部的 RAM，当任务越多时，需要使用的栈空间就越大，即需要使用的 RAM 空间就越多。一个 MCU 能够支持多少任务，就得看你的 RAM 空间有多少。

```

1. /* AppTaskCreate任务任务堆栈 */
2. static StackType_t AppTaskCreate_Stack[128];
3. /* LED任务堆栈 */
4. static StackType_t LED_Task_Stack[128];

```

2.4 定义任务控制块

定义好任务函数和任务栈之后，我们还需要为任务定义一个任务控制块，通常我们称这个任务控制块为任务的身份证。在 C 代码上，任务控制块就是一个结构体，里面有非常多的成员，这些成员共同描述了任务的全部信息，

```

1. /* AppTaskCreate 任务控制块 */
2. static StaticTask_t AppTaskCreate_TCB;
3. /* AppTaskCreate 任务控制块 */
4. static StaticTask_t LED_Task_TCB;

```

2.5 静态创建任务

一个任务的三要素是任务主体函数，任务栈，任务控制块，那么怎么样把这三个要素 联合在一起？FreeRTOS 里面有一个叫静态任务创建函数 xTaskCreateStatic()，它就是干这个活的。它将任务主体函数，任务栈（静态的）和任务控制块（静态的）这三者联系在一起，让任务可以随时被系统启动，

```
1.  /* 创建 AppTaskCreate 任务 */
2.  AppTaskCreate_Handle = xTaskCreateStatic((TaskFunction_t      )AppTaskCreate,          //任务函数
3.
4.                                          (const char*      )"AppTaskCr
5.                                          (uint32_t          )12
6.                                          (void*            )NU
7.                                          (UBaseType_t      )3,          //任
8.                                          (StackType_t*     )AppTaskCre
9.                                          (StaticTask_t*    )&AppTaskCr
10.
11.      if(NULL != AppTaskCreate_Handle)/* 创建成功 */
12.      vTaskStartScheduler(); /* 启动任务，开启调度 */
```



2.6 启动任务

当任务创建好后，是处于任务就绪（Ready），在就绪态的任务可以参与操作系统的调度。但是此时任务仅仅是创建了，还未开启任务调度器，也没创建空闲任务与定时器任务（如果使能了 configUSE_TIMERS 这个宏定义），那这两个任务就是在启动任务调度器中实现，每个操作系统，任务调度器只启动一次，之后就不会再次执行了，FreeRTOS 中启动任务调度器的函数是 vTaskStartScheduler()，并且启动任务调度器的时候就不会返回，从此任务管理都由 FreeRTOS 管理，此时才是真正进入实时操作系统中的第一步，

```
vTaskStartScheduler(); /* 启动任务，开启调度 */
```

2.7 main.c 文件内容全貌

现在我们把任务主体，任务栈，任务控制块这三部分代码统一放到 main.c 中，我们在 main.c 文件中创建一个 AppTaskCreate 任务，这个任务是用于创建用户任务，为了方便管理，我们的所有的任务创建都统一放在这个函数中，在这个函数中创建成功的任务就可以直接参与任务调度的，

```
1.  /* FreeRTOS 头文件 */
2.  #include "FreeRTOS.h"
3.  #include "task.h"
4.  /* 开发板硬件 bsp 头文件 */
5.  #include "bsp_led.h"
6.  #include "bsp_usart.h"
7.  static void AppTaskCreate(void); /* 用于创建任务 */
8.
9.  static void LED_Task(void* pvParameters); /* LED_Task 任务实现 */
10.
11. static void BSP_Init(void); /* 用于初始化板载相关资源 */
12. /***** 任务句柄 *****/
13. /*
14.  * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
15.  * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
16.  * 这个句柄可以为 NULL。
17.  */
18. /* 创建任务句柄 */
19. static TaskHandle_t AppTaskCreate_Handle;
20. /* LED 任务句柄 */
21. static TaskHandle_t LED_Task_Handle;
22.
23. /***** 内核对象句柄 *****/
24. /*
25.  * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
26.  * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
27.  * 们就可以通过这个句柄操作这些内核对象。
28.  */
29. 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
30. 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
31. 来完成的
32.  */
33.  */
34.
35.
36. /***** 全局变量声明 *****/
37. /*
38.  * 当我们在写应用程序的时候，可能需要用到一些全局变量。
39.  */
40. /* AppTaskCreate 任务任务堆栈 */
41. static StackType_t AppTaskCreate_Stack[128];
42. /* LED 任务堆栈 */
43. static StackType_t LED_Task_Stack[128];
44.
45. /* AppTaskCreate 任务控制块 */
46. static StaticTask_t AppTaskCreate_TCB;
47. /* AppTaskCreate 任务控制块 */
48. static StaticTask_t LED_Task_TCB;
49.
50. /* 空闲任务任务堆栈 */
51. static StackType_t Idle_Task_Stack[configMINIMAL_STACK_SIZE];
52. /* 定时器任务堆栈 */
53. static StackType_t Timer_Task_Stack[configTIMER_TASK_STACK_DEPTH];
54.
55. /* 空闲任务控制块 */
56. static StaticTask_t Idle_Task_TCB;
57. /* 定时器任务控制块 */
58. static StaticTask_t Timer_Task_TCB;
59.
60. /**
61.  * 使用了静态分配内存，以下这两个函数是由用户实现，函数在 task.c 文件中有引用
62.  * 当且仅当 configSUPPORT_STATIC_ALLOCATION 这个宏定义为 1 的时候才有效
63.  */
```

```

64. void vApplicationGetTimerTaskMemory(StaticTask_t **ppxTimerTaskTCBBuffer,
65.
66.
67.
68. void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer,
69.
70.
71.
72. /*****
73.  * @brief 主函数
74.  * @param 无
75.  * @retval 无
76.  * @note 第一步：开发板硬件初始化
77. 第二步：创建APP应用任务
78. 第三步：启动FreeRTOS，开始多任务调度
79. *****/
80. int main(void)
81. {
82.     /* 开发板硬件初始化 */
83.     BSP_Init();
84.     printf("这是一个[野火]-STM32全系列开发板-FreeRTOS-静态创建单任务!\r\n");
85.     /* 创建 AppTaskCreate 任务 */
86.     AppTaskCreate_Handle = xTaskCreateStatic((TaskFunction_t )AppTaskCreate, //任务函数
87.
88.
89.
90.
91.
92.
93.
94.     if(NULL != AppTaskCreate_Handle)/* 创建成功 */
95.     vTaskStartScheduler(); /* 启动任务，开启调度 */
96.
97.     while(1); /* 正常不会执行到这里 */
98. }
99.
100.
101. /*****
102.  * @ 函数名 : AppTaskCreate
103.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
104.  * @ 参数 : 无
105.  * @ 返回值 : 无
106. *****/
107. static void AppTaskCreate(void)
108. {
109.     taskENTER_CRITICAL(); //进入临界区
110.
111.     /* 创建LED_Task任务 */
112.     LED_Task_Handle = xTaskCreateStatic((TaskFunction_t )LED_Task, //任务函数
113.
114.
115.
116.
117.
118.
119.
120.     if(NULL != LED_Task_Handle)/* 创建成功 */
121.     printf("LED_Task任务创建成功!\n");
122.     else
123.     printf("LED_Task任务创建失败!\n");
124.
125.     vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
126.
127.     taskEXIT_CRITICAL(); //退出临界区
128. }
129.
130.
131.
132. /*****
133.  * @ 函数名 : LED_Task
134.  * @ 功能说明: LED_Task任务主体
135.  * @ 参数 :
136.  * @ 返回值 : 无
137. *****/
138. static void LED_Task(void* parameter)
139. {
140.     while (1)
141.     {
142.         LED1_ON;
143.         vTaskDelay(500); /* 延时500个tick */
144.         printf("LED_Task Running, LED1_ON\r\n");
145.
146.         LED1_OFF;
147.         vTaskDelay(500); /* 延时500个tick */
148.         printf("LED_Task Running, LED1_OFF\r\n");
149.     }
150. }
151.
152. /*****
153.  * @ 函数名 : BSP_Init
154.  * @ 功能说明: 板级外设初始化，所有板子上的初始化均可放在这个函数里面
155.  * @ 参数 :
156.  * @ 返回值 : 无
157. *****/
158. static void BSP_Init(void)
159. {
160.     /*
161.     * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
162.     * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，
163.     * 都统一用这个优先级分组，千万不要再分组，切忌。
164.     */
165.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
166.
167.     /* LED 初始化 */

```

Sta
uin

StackType_
uint32_t *

(const char*)"AppTaskCr
(uint32_t)12
(void*)NU
(UBaseType_t)3, //
(StackType_t*)AppTaskCre
(StaticTask_t*)&AppTaskCr

(const char*)"LED_Task"
(uint32_t)12
(void*)NU
(UBaseType_t)4,
(StackType_t*)LED_Task_S
(StaticTask_t*)&LED_Task_

```

168.         LED_GPIO_Config();
169.
170.         /* 测试硬件是否正常工作 */
171.         LED1_ON;
172.
173.         /* 让程序停在这里，不再继续往下执行 */
174.         while(1);
175.
176.         /* 串口初始化 */
177.         USART_Config();
178.
179.     }
180.
181.
182. /**
183.  *****
184.  * @brief  获取空闲任务的任务堆栈和任务控制块内存
185.  *
186.  *          ppxTimerTaskTCBBuffer :          任务控制块内存
187.  *          ppxTimerTaskStackBuffer :        任务堆栈内存
188.  *          pulTimerTaskStackSize :          任务堆栈大小
189.  * @date    2018-xx-xx
190.  *****
191.  */
192. void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer,
193.                                     StackType_t **ppxIdleTaskStackBuffer,
194.                                     uint32_t *pulIdleTaskStackSize)
195. {
196.     *ppxIdleTaskTCBBuffer=&Idle_Task_TCB; /* 任务控制块内存 */
197.     *ppxIdleTaskStackBuffer=Idle_Task_Stack; /* 任务堆栈内存 */
198.     *pulIdleTaskStackSize=configMINIMAL_STACK_SIZE; /* 任务堆栈大小 */
199. }
200. /**
201.  *****
202.  * @brief  获取定时器任务的任务堆栈和任务控制块内存
203.  *
204.  *          ppxTimerTaskTCBBuffer :          任务控制块内存
205.  *          ppxTimerTaskStackBuffer :        任务堆栈内存
206.  *          pulTimerTaskStackSize :          任务堆栈大小
207.  *****
208.  */
209. void vApplicationGetTimerTaskMemory(StaticTask_t **ppxTimerTaskTCBBuffer,
210.                                     StackType_t **ppxTimerTaskStackBuffer,
211.                                     uint32_t *pulTimerTaskStackSize)
212. {
213.     *ppxTimerTaskTCBBuffer=&Timer_Task_TCB; /* 任务控制块内存 */
214.     *ppxTimerTaskStackBuffer=Timer_Task_Stack; /* 任务堆栈内存 */
215.     *pulTimerTaskStackSize=configTIMER_TASK_STACK_DEPTH; /* 任务堆栈大小 */
216. }
217. /*****END OF FILE*****/

```



注意：在使用静态创建任务的时候必须要将 FreeRTOSConfig.h 中的 configSUPPORT_STATIC_ALLOCATION 宏配置为 1。

下载后，可以看到板子上的 LED 灯已经在 闪烁，说明我们创建的单任务（使用静态内存）已经跑起来了。

本篇内容摘自野火《FreeRTOS内核实现与应用开发》